

DD: FUL-02 — Order Capture (Parent)

Developer implementation guide: DD_FUL-02-Order_Capture-DEV_IMPL_GUIDE-v1.0.md.

MVP simplification baseline: first release implements the WIK revenue-critical happy path only: capture, validate, KYC, payment, installation, subscription creation, activation trigger, success notification, and cancellation/timeout handling. Other operator BPMNs, rare branches, welcome-call variants, and advanced compensation paths are phased after this path is stable.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_FUL-02-Order_Capture-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 Order capture and onboarding orchestration for new subscriptions. Camunda BPMN-driven per FOUNDATION_CAMUNDA's reservation of Camunda for installation/ticketing/work-order processes, per SUB-WF-FRAMEWORK-01 v0.9. **This is the parent DD** of a 12-DD constellation. It carries the cross-step narrative, glossary, the satellite index, cross-DD coordination, and the KE workflow / TZ codebase coverage / Gap & V3 design rationale. Detailed contracts (worker contracts, message catches, user tasks, APIs, events, rules, schema columns touched) live in the step satellites and the framework satellite indexed in §3 below. **Scope is strictly new-customer onboarding.** MACD operations on existing subscriptions (Move / Add / Change / Disconnect) remain owned by SUB-WF-* workflows.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/06_fulfillment/DD_FUL-02-Order_Capture-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- POST /activate
- POST /api/orders
- POST /api/orders/{id}/cancel

- POST /api/subscriptions
- POST /api/work-orders
- POST /engine-rest/message

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- fulfillment-activation
- fulfillment-order-capture
- kenya-kyc-leaders
- kenya-l1-kyc
- kyc-l1-supervisor-approval
- kyc-team-leader-approval
- order-cascade-cancellation
- order-compute-expected-payment
- order-create-install-wo
- order-create-subscription
- order-mark-activation-failed
- order-mark-cancelled
- order-mark-install-failed
- order-mark-rejected
- order-notify-failure
- order-notify-payment-instructions
- order-notify-success
- order-trigger-activation
- order-validate
- wait-install-finalize
- wait-payment

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- PaymentApplied

- PaymentAppliedConsumer
- PaymentReceived
- WorkOrderCancelled
- WorkOrderCancelledConsumer
- WorkOrderFinalized
- WorkOrderFinalizedConsumer

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0

Order capture and onboarding orchestration for new subscriptions. Camunda BPMN-driven per FOUNDATION_CAMUNDA's reservation of Camunda for installation/ticketing/work-order processes, per SUB-WF-FRAMEWORK-01 v0.9.

This is the parent DD of a 12-DD constellation. It carries the cross-step narrative, glossary, the satellite index, cross-DD coordination, and the KE workflow / TZ codebase coverage / Gap & V3 design rationale. Detailed contracts (worker contracts,

message catches, user tasks, APIs, events, rules, schema columns touched) live in the step satellites and the framework satellite indexed in §3 below.

Scope is strictly new-customer onboarding. MACD operations on existing subscriptions (Move / Add / Change / Disconnect) remain owned by SUB-WF-* workflows.

1. What this process is about

A customer expresses intent to subscribe to a Wananchi Kenya, Wananchi Uganda, Wananchi Tanzania, or Yas Senegal Package at a known HomePass. The capturing channel (Field Agent PWA, Backoffice UI, Call Center, self-service portal) calls FUL-02's POST /api/orders. FUL-02 writes a row into customer_order and then asks **Camunda** — a workflow engine that lives in its own server outside our module, like a separate database for orchestration — to start running the operator's order-capture flow for this customer.

The "operator's flow" is defined in a **BPMN file** — a flowchart-like XML document that lists the steps an order goes through, in what order, with which branches. BPMN files live inside our module's resources folder (modules/fulfillment-order-capture/src/main/resources/workflows/) and get deployed into Camunda at module startup. Once Camunda has the BPMN, it can execute it. Each running execution is called a **process instance** — think of the BPMN as a recipe and the process instance as one specific meal being cooked from that recipe. Each order has exactly one process instance running for it.

From that point on, **Camunda drives the order**. Camunda doesn't do the actual work itself — it just keeps track of where the order is in the flow and tells our code "OK, now it's time to validate this order" or "now it's time to create the subscription." When that happens, our code does the work and tells Camunda the result. The pieces of our code that do the actual work are called **workers**. A worker is just a Java bean living inside our module that asks Camunda "anything for me to do?" on a loop, picks up tasks Camunda hands out, runs them, and reports the result back. We have 16 such workers in v1.0 — examples: order-validate checks the order is valid, order-create-subscription calls SUB-LM-01 to create the subscription, order-trigger-activation calls SUB-WF-ACTIVATE-01 to fire activation. The BPMN step where Camunda hands out work to a worker is called an **external task** (because the work happens outside Camunda's own process — in our module). When the recipe (BPMN) says "do validation here," Camunda creates an external task with topic order-validate; the worker subscribed to that topic picks it up, does the work, completes. Camunda then moves the recipe forward to the next step.

Some steps in the flow need a human, not code. For those, the BPMN has a **user task** — a step that just sits there and waits for someone to claim it and finish it through a UI. In v1.0 we have 3 user tasks: KYC L1 supervisor approval, KYC Team Leader approval, Welcome Call agent task. Each user task carries an **HTTP connector** — a built-in Camunda feature that fires an HTTP POST when the task is created, so we can notify the responsible team that there's work waiting for them. (The connector calls back into our own module via NotificationHandlerResource, which then dispatches to email/SMS/Slack/Teams.)

Some steps in the flow need to **wait** for an external thing to happen — a payment to be received, an install WO to be finalized. For those, the BPMN has a **message catch event** — a step that parks the process instance and goes to sleep until someone notifies Camunda "hey, the thing happened." The notification comes via Camunda's REST API (POST /engine-rest/message). Our code calls that endpoint when, for example, a Kafka consumer in our module sees PaymentApplied arrive on the bus — the consumer calls POST /engine-rest/message with messageName=PaymentReceived and the order's ID; Camunda finds the parked instance and wakes it up. We have 4 such messages in v1.0: PaymentReceived, WorkOrderFinalized, WorkOrderCancelled, OrderCancellationRequested.

Waiting can't be forever. So a message catch event is usually paired with a **boundary timer** — a side-step attached to the wait that fires after a configured duration (PT7D = 7 days, PT30D = 30 days, etc.). If the message arrives first, Camunda wakes the instance and proceeds normally; if the timer fires first, Camunda interrupts the wait and routes to a failure path (typically a mark - * - failed worker). v1.0 has 2 such timers: PT7D on payment wait (KE WIK), PT30D on install wait.

Two more concepts. **businessKey** is how Camunda knows which order a process instance belongs to — we set businessKey = orderId when starting the instance, then we can look up the instance later by that key (e.g., when we need to send it a message). And **Drools KJAR** is a separate engine for decision rules — a .jar file holding business rules written in DRL (Drools Rule Language). We use it for operator-specific gateway decisions in the BPMN. The worker preceding a BPMN gateway calls Drools, gets back a decision (a process variable), and the BPMN gateway routes on that variable. Keeps operator-specific business rules out of code and out of BPMN, owned by business analysts.

v1.0 ships the Wananchi Kenya shape: door-to-door capture + two-level KYC + pay-first + one install WO + activation. The architecture supports operator-specific BPMN compositions (different sequences for different operators) without code forks — see §2.2 for the architecture note. The condensed narrative in §2.1 walks the KE shape; each step satellite carries step-local

detail.

The current operator scope is **WIK, WUG, WTZ, YASSN**. Adding a future operator: drop the operator's BPMN file, add a `ful_02_config` row, define operator-scoped Drools rules and FOUNDATION_AUTH candidate groups. No new worker code in the common case.

What FUL-02 does not own

- **Customer master record + KYC documents** — owned by future ILM-CFG-01. v1.0 uses a `customer_stub`.
- **Subscription state machine** — owned by SUB-LM-01.
- **Activation saga** — owned by SUB-WF-ACTIVATE-01.
- **Service provisioning at the network level** — owned by FUL-03.
- **Work Order machinery** — owned by the future WO module DD. v1.0 uses `install_wo_stub` (supports `kind=install` only in v1.0; future WO module DD will own the full WO state machine and may add additional kinds).
- **Payment processing** — owned by BIL-01-PAY-01.
- **Refund on cancellation** — owned by BIL-02-ADJ-01 + BIL-01-PAY-01.
- **Customer-facing notifications** — owned by DD_NOT-01.
- **Staff-facing notifications** — owned by future ICN-01. v1.0 uses `staff_notification_stub`.
- **MACD on existing subscriptions** — owned by SUB-WF-* workflows.

Glossary (quick reference)

For convenience after the prose above, here's the same set of concepts in table form:

Term	What it is in plain language	Where it lives
Order	A customer's intent to subscribe to a Package at a HomePass — the thing being orchestrated. A row in <code>customer_order</code> , linked one-to-one to a Camunda process instance via <code>process_instance_id</code> .	Our module's database
BPMN file	A flowchart-like XML document defining the order-capture flow per operator (<code>OrderCapture_WIK.bpmn</code> , etc.). The recipe.	Our module's resources; deployed into Camunda at startup
Process instance	A live execution of a BPMN file for one specific order. The meal being cooked from the recipe.	Camunda
businessKey	The order's ID, set on the process instance so we can find it again.	Camunda
External task	A BPMN step that hands work out to our module's code.	Defined in BPMN; executed by our worker
Worker	Our Java code that long-polls Camunda for external tasks, runs the work, reports back. 16 in v1.0.	Our module
User task	A BPMN step waiting for a human, not code. Fires an HTTP notification on creation via the Camunda built-in HTTP connector. 3 in v1.0.	Defined in BPMN; completed via UI → our module's REST endpoint → Camunda
Message catch event	A BPMN step that parks the instance until someone calls Camunda's <code>POST /engine-rest/message</code> with the matching message name + <code>businessKey</code> . 5 in v1.0.	Defined in BPMN; awakened by our Kafka consumers or our cancel API
Boundary timer	A side-step attached to a waiting step, firing after a duration (PT7D, etc.) if the wait doesn't end. Used to enforce timeouts.	Defined in BPMN; managed by Camunda's job executor
Process variable	A typed key-value carried with the process instance (small payloads only — large state lives in <code>customer_order</code>).	Camunda
Drools KJAR	A separate engine holding operator-specific business rules in DRL. Workers call it before a gateway when an operator-specific decision is needed.	Drools KIE Server (separate from Camunda); rules deployed as <code>.jar</code> files

2. Condensed runtime narratives

Two complete order journeys at a glance. Each step links to its satellite for full detail.

2.1 Wananchi Kenya shape — door-to-door, pay-first, two-level KYC

The agent captures the customer's details on the Field Agent PWA and taps "Submit Order" at 09:00. The PWA POSTs to FUL-02 → **STEP-CAPTURE**: the row is written into `customer_order`, Camunda starts the process instance for the BPMN `OrderCapture_WIK`.

The instance immediately advances to validation → **STEP-VALIDATE**: the `order-validate` worker (the Java code that checks whether the order can proceed — HomePass sellable, Package active, tech regions match, no duplicate subscription, customer eligible) runs the 5-check suite. All pass.

Camunda routes to subscription creation → **STEP-SUBSCRIPTION**: the `order-create-subscription` worker (the Java code that asks SUB-LM-01 to create the actual subscription record) calls SUB-LM-01 POST `/api/subscriptions`, gets back `sub_01HX...A`.

Then to payment computation and notification → **STEP-PAYMENT**: two workers run back-to-back. First `order-compute-expected-payment` (figures out how much the customer needs to pay — installation fee + first cycle prorate, looked up from BIL-CFG-01's `BillableEvent` catalog) resolves 15000 KES. Then `order-notify-payment-instructions` (just emits a Kafka event saying "this order is awaiting payment, the customer should be told how to pay") emits `sophix.orders.awaiting-payment`; DD_NOT-01 picks that event up and sends the customer the M-Pesa paybill instructions via SMS+email. The BPMN now parks at a `wait-payment` step (a message catch — a place the recipe goes to sleep until "payment received" arrives), guarded by a PT7D boundary timer (a 7-day fuse — if the customer hasn't paid by then, the BPMN routes to cancellation).

At 11:23 the customer pays. BIL-01-PAY-01 (the billing module's payment application service) emits `PaymentApplied` on Kafka. `PaymentAppliedConsumer` (our module's Kafka consumer for that event — Java code subscribed to the topic, runs in our module) sees the event, looks up which order it belongs to (by `subscription_id`), then calls Camunda's POST `/engine-rest/message` to wake the parked instance with message name `PaymentReceived`. Camunda wakes the BPMN.

The BPMN advances to the first KYC user task → **STEP-KYC**: a user task is a step that waits for a human. Camunda creates `kyc-l1-supervisor-approval` and fires its HTTP connector (the built-in Camunda feature for notifying when a user task appears — see §1) at our `NotificationHandlerResource` endpoint, which dispatches an email/SMS/Slack to the L1 supervisor (candidate group `kenya-l1-kyc` in `FOUNDATION_AUTH`). The supervisor opens BO UI, reviews, clicks Approve; BO UI calls our completion endpoint which forwards the completion to Camunda. The BPMN advances to the next user task `kyc-team-leader-approval`, dispatched to `kenya-kyc-leaders`. Same flow. KYC complete.

Camunda routes to install WO creation → **STEP-INSTALL**: the `order-create-install-wo` worker (Java code that resolves equipment requirements via PLM-CFG-01 — for this triple-play package: GPON_ONT + ATA — then asks the WO module to create the install Work Order; v1.0 uses an `install_wo_stub` table because the WO module DD isn't built yet) creates the install WO, emits `OrderInstallDispatched`. The BPMN parks at `wait-install-finalize` (another message catch) with a PT30D boundary (30-day fuse). Three hours later the contractor finalizes the install; the WO module emits `WorkOrderFinalized` on Kafka; `WorkOrderFinalizedConsumer` (the matching Kafka consumer in our module) wakes the BPMN by calling Camunda's `/engine-rest/message`.

Camunda routes to activation → **STEP-ACTIVATION**: the `order-trigger-activation` worker calls SUB-WF-ACTIVATE-01 to start activation and receives an operation id quickly. FUL-02 must not hold an HTTP call open while the full activation saga runs. The BPMN waits for the `activation-completed/failed` event correlated by order id, subscription id, and operation id. After completion, the `order-notify-success` worker sets `current_phase = ACTIVATED`, emits `sophix.orders.activated`; DD_NOT-01 sends the customer the "service is live" SMS+email. Camunda closes the process instance.

Total elapsed: 5h 25min. Active worker + Camunda time: under 30s — the rest is the customer paying (2h) + install dispatch wait (3h).

2.2 Architecture note — other operator shapes are supportable

v1.0 ships the KE WIK shape (§2.1). The architecture is deliberately operator-agnostic at the worker level: workers are primitives (e.g., `order-validate`, `order-create-subscription`, `order-compute-expected-payment`, `order-create-install-wo`, `order-trigger-activation`) that don't know what shape they're running in — they read process variables, do their work, report back. The shape comes from the BPMN.

A different operator could therefore compose the same primitives differently — for example, a deployment that wants a welcome-call qualification step before payment, or one that wants to defer subscription creation until a feasibility survey confirms the install can happen. Such a deployment would (a) write its own `OrderCapture_{OPERATOR}.bpmn`, (b) potentially add new worker types or user-task types as new step satellites, (c) add operator-specific candidate

groups in FOUNDATION_AUTH. The existing workers stay unchanged.

v1.0 does NOT ship any of those alternative shapes. They are mentioned only to confirm the architecture supports them as future work, without that future work requiring rewrites of FUL-02's existing v1.0 deliverables.

2.3 Cancellation applies across the journey

A cancellation may be requested at any phase before ACTIVATED → **STEP-CANCELLATION**: when the cancel API is hit (or the PT7D payment timer fires), Camunda gets a message and the BPMN's top-level **interrupting boundary event** fires — a fancy way of saying the recipe has an emergency stop attached to the whole flow. It interrupts whatever step was running. Camunda then routes to order - cascade - cancellation (a worker that reads what state the order reached and undoes downstream things one by one: terminate the subscription if one was created, cancel the install WO if it exists, emit a refund request if money was received).

Cross-step mechanisms — Camunda primitives, the cancellation pattern, phase projection, per-operator BPMN composition rules, the notification handler endpoint contract, the shared schemas — live in the **FRAMEWORK** satellite.

3. Satellite index

Each satellite is self-contained per memory rule 11. A developer working on a satellite needs only that satellite + FRAMEWORK to implement their slice.

Satellite	File	Owns	Phases	Lines (target)
FRAMEWORK	DD_FUL-02-FRAMEWORK-v1.0.md	Camunda primitives (start, fetchAndLock, complete, message, XML patterns, deployment lifecycle), cancellation pattern, phase projection mechanism, per-operator BPMN composition rules, notification handler endpoint contract, shared schemas (customer_order, customer_order_phase_history, ful_02_config), v1.0 stubs (customer_stub, install_wo_stub, staff_notification_stub)	—	~400
STEP-CAPTURE	DD_FUL-02-STEP-CAPTURE-v1.0.md	POST /api/orders endpoint, idempotency rule, BPMN start mechanism, OrderCreated event	CAPTURED	~180
STEP-VALIDATE	DD_FUL-02-STEP-VALIDATE-v1.0.md	order-validate worker, 5-check matrix, order-mark-rejected worker, OrderValidated / OrderRejected events	IN_VALIDATION → REJECTED (failure)	~150
STEP-SUBSCRIPTION	DD_FUL-02-STEP-SUBSCRIPTION-v1.0.md	order-create-subscription worker, call to SUB-LM-01, subscription_id column lifecycle	(no dedicated phase — transitive)	~120
STEP-PAYMENT	DD_FUL-02-STEP-PAYMENT-v1.0.md	order-compute-expected-payment + order-notify-payment-instructions workers, PaymentReceived message catch + PaymentAppliedConsumer (side-channel correlation), PT7D boundary timer, partial-payment handling, OrderAwaitingPayment event	AWAITING_PAYMENT	~250
STEP-KYC	DD_FUL-02-STEP-KYC-v1.0.md	kyc-l1-supervisor-approval + kyc-team-leader-approval user tasks, kenya-l1-kyc + kenya-kyc-leaders candidate groups, decision outcomes per level	IN_KYC	~150

Satellite	File	Owens	Phases	Lines (target)
STEP-INSTALL	DD_FUL-02-STEP-INSTALL-v1.0.md	order-create-install-wo worker, WorkOrderFinalized + WorkOrderCancelled message catches + consumers, PT30D boundary, order-mark-install-failed worker, equipment bindings flow, OrderInstallDispatched event	READY_FOR_INSTALL → INSTALLING → INSTALL_FAILED (failure)	~250
STEP-ACTIVATION	DD_FUL-02-STEP-ACTIVATION-v1.0.md	order-trigger-activation worker (95s lock), call to SUB-WF-ACTIVATE-01, order-mark-activation-failed worker, order-notify-success worker, OrderActivated / OrderActivationFailed events	ACTIVATING → ACTIVATED or ACTIVATION_FAILED	~200
STEP-CANCELLATION	DD_FUL-02-STEP-CANCELLATION-v1.0.md	POST /api/orders/{id}/cancel endpoint, OrderCancellationRequested message (API-sent, no Kafka), order-cascade-cancellation worker, order-mark-cancelled worker, OrderCancelled event	CANCELLED	~150

Total: 1 parent + 1 framework + 8 steps = 10 DDs. Each step DD owned by one dev. FRAMEWORK + parent shared/coordinator-owned.

Worker → satellite map

For a reader asking "which satellite owns worker X" — and what does X actually do:

Worker	What it does	Satellite
order-validate	Checks the order is valid (HomePass sellable, Package active, regions match, no duplicate, customer eligible)	STEP-VALIDATE
order-create-subscription	Asks SUB-LM-01 to create the subscription record	STEP-SUBSCRIPTION
order-compute-expected-payment	Figures out how much the customer needs to pay (install fee + first cycle prorated)	STEP-PAYMENT
order-notify-payment-instructions	Emits the "awaiting payment" Kafka event so DD_NOT-01 can tell the customer how to pay	STEP-PAYMENT
order-create-install-wo	Resolves equipment requirements via PLM-CFG-01, asks the WO module to create the install WO	STEP-INSTALL
order-trigger-activation	Calls SUB-WF-ACTIVATE-01's activation saga synchronously (the only long-running worker, up to 95s)	STEP-ACTIVATION
order-cascade-cancellation	On cancellation, unwinds downstream state: terminate subscription if exists, cancel install WO if exists, emit refund request if money was paid	STEP-CANCELLATION
order-mark-rejected	Terminal worker for validation failures — sets phase REJECTED, emits the rejection event	STEP-VALIDATE
order-mark-cancelled	Terminal worker for cancellation — sets phase CANCELLED, emits the cancelled event	STEP-CANCELLATION
order-mark-activation-failed	Terminal worker for activation failures — sets phase ACTIVATION_FAILED, emits the failure event	STEP-ACTIVATION
order-mark-install-failed	Terminal worker for install failures (timeout, WO cancelled) — sets phase INSTALL_FAILED, emits the failure event	STEP-INSTALL
order-notify-success	Terminal happy-path worker — sets phase ACTIVATED, emits the success event so DD_NOT-01 can send the "service is live" notification	STEP-ACTIVATION

Worker	What it does	Satellite
order-notify-failure	Generic terminal failure worker for operator BPMNs that want a single fallback path	FRAMEWORK

Message catch → satellite map

A message catch is a BPMN step that parks the order and waits for an external signal. Each row below says what wakes the parked order:

Message	What wakes it	Satellite
PaymentReceived	Our PaymentAppliedConsumer (Kafka consumer for BIL-01's PaymentApplied) calls Camunda's message endpoint after a payment is applied	STEP-PAYMENT
WorkOrderFinalized	Our WorkOrderFinalizedConsumer (Kafka consumer for sophix.workorders.finalized) — the contractor finalized the install	STEP-INSTALL
WorkOrderCancelled	Our WorkOrderCancelledConsumer — the install WO was cancelled (contractor unable, operator-initiated, cascade)	STEP-INSTALL
OrderCancellationRequested	Our cancel API endpoint (calls Camunda's message endpoint directly, no Kafka in the middle)	STEP-CANCELLATION

User task → satellite map

A user task is a BPMN step waiting for a human, dispatched via HTTP connector to the responsible team:

User task	Who picks it up	Satellite
kyc-l1-supervisor-approval	L1 supervisor in candidate group kenya-l1-kyc (BO UI)	STEP-KYC
kyc-team-leader-approval	Wananchi Team Leader in candidate group kenya-kyc-leaders (BO UI)	STEP-KYC

4. Cross-DD coordination

4.1 Module owner and impacted modules

Role	Module	What it does
Owner	New fulfillment-order-capture module (Java/Spring; fulfillment cluster — separate from fulfillment-activation which owns FUL-03). Owns: REST API; BPMN files in src/main/resources/workflows/; the 16 external task workers; the 4 Kafka consumers; the user-task notification handler endpoints; the schemas in FRAMEWORK; the v1.0 stubs.	
Impacted (callees via workers)	ILM-CFG-01 (pending; v1.0 stub) — customer lookup. SIP-01 — Package read. RLM-CFG-01 — HomePass read. SUB-LM-01 — Subscription create. WO module (pending; v1.0 stub) — install WO create. BIL-CFG-01 — BillableEvent lookup. BIL-01-PAY-01 — mark-paid-offline. BIL-02-ADJ-01 — refund. SUB-WF-ACTIVATE-01 — activation trigger. SUB-WF-TERMINATE-01 — cancellation cleanup. Camunda /engine-rest. Drools KIE Server.	
Impacted (callers)	Field Agent PWA. Backoffice Dispatch UI. Call Center UI. Self-service portal.	
Impacted (consumers)	DD_NOT-01 (customer-facing comms only). ICN-01 (staff-facing comms — pending DD). CVM (acquisition funnel). Finance ETL. NOC alerting.	
Engines	Camunda (BPMN orchestration). Kafka. Drools (operator-scoped KJARs).	

4.2 Boundary clarifications

Touched	What FUL-02 owns	What the other DD owns
FOUNDATION_CAMUNDA	Hosts operator BPMNs. Workers long-poll. Message correlation. Variable reads.	Engine, deployment versioning, message correlation infra, Job Executor for timers, history.
FOUNDATION_DROOLS	Operator-scoped KJARs evaluated by workers when BPMN gateways need operator-specific decisions.	DRL evaluation.
ILM-CFG-01 (pending)	order-validate worker calls customer service. v1.0 customer_stub.	Customer master + KYC.
SIP-01 Package Management	order-validate reads Package; cached.	Package catalog.
RLM-CFG-01 HomePass Configuration	order-validate reads HomePass; cached.	HomePass catalog.
SUB-LM-01	order-create-subscription calls POST /api/subscriptions. order-cascade-cancellation calls SUB-WF-TERMINATE-01.	Subscription state.
WO module (pending)	order-create-install-wo calls WO module. Kafka consumers correlate WorkOrderFinalized / WorkOrderCancelled. v1.0 install_wo_stub + admin finalize endpoints.	WO routing, FINALIZE workflow.
BIL-CFG-01 BillableEvent Catalog	order-compute-expected-payment reads BillableEvents.	BillableEvent catalog.
BIL-01-PAY-01 Payment Application	PaymentAppliedConsumer correlates PaymentApplied → PaymentReceived. Admin endpoint routes mark-paid-offline through this DD.	Payment receipt.
BIL-02-ADJ-01 Invoice Adjustments	order-cascade-cancellation emits OrderRefundRequested for operator-driven refund.	Adjustments.
SUB-WF-FRAMEWORK-01	Reservation of Camunda for installation/work-order flows.	Workflow framework.
SUB-WF-ACTIVATE-01	order-trigger-activation calls POST /activate.	Activation saga.
SUB-WF-TERMINATE-01	Called by cancellation cascade.	Termination saga.
DD_NOT-01	Consumes order events for customer-facing SMS/email. NOT called for staff comms — strictly customer-scoped.	Customer notification dispatch.
ICN-01 Internal Communications (pending DD)	Notification handler calls ICN-01 for staff-task notifications. v1.0 staff_notification_stub.	Staff notification dispatch.
FUL-03 Service Activation	NOT called directly. Invoked transitively via SUB-WF-ACTIVATE-01.	Service activation.

4.3 Dependencies

Dependency	Owner	Status
FOUNDATION_CAMUNDA	infra team	✓
FOUNDATION_DROOLS	infra team	✓
FOUNDATION_KAFKA	infra team	✓
FOUNDATION_AUTH	infra team	✓
ILM-CFG-01 Customer Service	ilm team	■ pending (v1.0 stub customer_stub)
SIP-01 Package Management	plm team	✓ v1.0
RLM-CFG-01 HomePass Configuration	network/plm team	✓ v1.0
SUB-LM-01 Subscription Lifecycle	subscription team	✓ v0.8
SUB-WF-FRAMEWORK-01	subscription team	✓ v0.9
SUB-WF-ACTIVATE-01	subscription team	✓ v0.9
SUB-WF-TERMINATE-01	subscription team	✓ v0.9
BIL-CFG-01 BillableEvent Catalog	billing team	✓ v0.8
BIL-01-PAY-01 Payment Application	billing team	✓ v1.0
BIL-02-ADJ-01 Invoice Adjustments	billing team	✓ v1.3
WO module	fulfillment team	■ pending (v1.0 stub install_wo_stub)

Dependency	Owner	Status
ICN-01 Internal Communications	platform team	■ pending (v1.0 stub staff_notification_stub)
FUL-03 Service Activation	fulfillment team	✓ v1.0 (called transitively)
DD_NOT-01 Notification Service	notification team	✓ v1.3 (customer-facing only)

5. KE workflow target · TZ codebase coverage · Gap & V3 design rationale

5.1 KE workflow target

Workshop sources: Customer Management deep dive (2026-04-27 / 2026-04-28); Provisioning deep dive (2026-04-30). These workshops capture how Wananchi Kenya needs to capture orders for the V3 redeploy.

Order capture workflow Wananchi Kenya needs:

- Door-to-door capture by franchise agent on a **Field Agent PWA**; KYC documents + Package selection at the customer's door.
- Customer record creation up-front (V3 first-class customer entity; the legacy KE BSS uses a dummy-account-from-pool with RPA pre-population because BroadHub has no account creation API — a model V3 explicitly drops).
- **KYC two-level approval** mandatory before service dispatch: L1 supervisor (regional / franchise) → final approval by Wananchi Team Leader. Future IPRS integration for OCR-driven automation.
- **Package Assignment** triggering auto-derivation of Services and creation of an install Work Order.
- **Pay-first invariant**: contractor not dispatched until customer payment is received.
- **Work Order machinery** (workshop verbatim — what Wananchi KE operationally needs):
 - One WO per assignment event — created systematically.
 - Equipment-delta rule: no equipment delta = audit-trail-only WO; delta ≠ 0 = contractor WO.
 - WO routing: BO → Contractor Slot (auto-assigned by Tech Region from HomePass + WO scope + availability) → Team Slot (contractor picks own team) → contractor Dispatcher FINALIZES → commands pushed to network through NMS.
 - Basic WO catalog (clone-extendable): Activation, Change of Service, Change Bill Cycle, Installation, Follow-Up.
 - Equipment follows the WO: same region/tech → equipment kept; otherwise re-attached to customer at FINALIZE.
- **Multi-DB observation** from workshop (verbatim): same client supports multiple operator databases ("Database: Sibanand Trading alongside Zuku Kenya"). Open workshop question (verbatim): "relationships with other fiber operators in the group (Uganda, Zambia, ...): shared instance vs separate per-country databases · cross-tenant data visibility · shared catalog vs per-country catalog."* V3 scope covers WIK/WUG/WTZ/YASSN only.

Open KE-workflow questions from workshop (per memory rule 7 — captured, not invented):

- Manual resume after auto-suspend — allowed? Same prior package or specific reactivation package?
- Move WO linkage — parent-child between Shifting Disconnect and Shifting Reconnect, or independent WOs sharing customer reference?
- Address-mismatch caught at deployment step (installer confirms with customer); two cases: nearby OLT with port → resolve; no OLT → refund / wait.
- ■ Call Center turnover flagged as high-risk for full CC agent account access; scoped read-only view + audited access pattern recommended for redesign.

Workflow areas not workshop-covered (to be specified in V3 by other means):

- Order state machine at the legacy KE BSS level (no first-class order entity exists there — V3 introduces it).
- Self-service portal flow specifics (existing customer app: view-only daily-refresh; V3 target: real-time direct order capture).
- SLA on the order pipeline (workshop noted SLA evaluation lives outside the current KE BSS; V3 candidate brings SLA inside SOPHIX — future FUL-OPS-SLA DD).
- Multi-country relationship topology (V3 scope is WIK/WUG/WTZ/YASSN only).

5.2 TZ codebase coverage of KE workflow today

To be filled in after codebase review. No claims about what the TZ-deployed Sophix codebase does for order capture are recorded here until the codebase has been read directly.

The cartography phase (per the pending plan) will produce this content. Once the codebase is available, this section will be filled with verified statements about which order-capture-related capabilities exist and which do not, including the actual state model, the enum definitions, the integration patterns with billing modules, and the equivalent of the V3 phase entities — replacing this placeholder.

5.3 V3 design rationale

V3's design choices follow from the KE workflow target (§5.1) and the V3 plan to support all four operating countries (WIK/WUG/WTZ/YASSN) from one code base. The design intent:

What the V3 design introduces	What it brings	Why
Per-operator workflow variants without code forks	Camunda BPMN per operator (OrderCapture_WIK.bpmn, etc.) + operator-agnostic external task workers (FRAMEWORK §1–§2). v1.0 ships KE WIK only; UG/TZ/SN BPMNs are pending future workshops	One module, four operators. The BPMN per operator chooses the sequence; the same workers run for any operator
Workflow as deployable artifact owned by business analysts, not buried in code	BPMN files in modules/fulfillment-order-capture/src/main/resources/workflows/; redeployable without rebuilding the module (FRAMEWORK §1.10)	BA + workflow analysts adjust BPMN with deployment versioning preserving in-flight instances
Declarative async waits with timeouts (PT7D payment, PT30D install)	Camunda message catch events with boundary timers (FRAMEWORK §1.7)	Timers are first-class, replayable, auditable in Camunda history rather than enforced via separate scanner jobs
Structured cancellation that unwinds downstream state uniformly	Interrupting boundary message event on top-level subprocess + conditional cascade worker (FRAMEWORK §1.8, STEP-CANCELLATION)	One cancellation entry point that branches on what state was reached, vs scattered per-state cancellation paths
Real-time event stream for downstream consumers (CVM funnel, NOC SLA, Finance forecasting, DD_NOT-01 customer comms, ICN-01 staff comms)	Kafka events emitted per step (every step satellite §5) — OrderCreated, OrderValidated, OrderAwaitingPayment, OrderPaymentReceived, OrderKycAwaitingL1, OrderInstallDispatched, OrderActivated, etc.	Cross-module consumers don't need to poll or webhook; they subscribe to the order journey on the bus
Customer-vs-staff notification separation	DD_NOT-01 for customer (strict); ICN-01 (pending) for staff; NotificationHandlerResource + staff_notification_stub (FRAMEWORK §3)	Cleanly partitioned model to avoid customer-channel and staff-channel notifications mixing
Phase projection for UI / dashboards / coarse-grained ops view	customer_order.current_phase projected from Camunda activity via per-operator activity_phase_map config; customer_order_phase_history partitioned table (FRAMEWORK §4, §5.2)	Ops needs an aggregate phase view ("how many orders are AWAITING_PAYMENT right now?") without grepping audit logs
Operator-specific gateway decisions	Drools KJAR per operator, called from a preceding worker that sets a Camunda variable; BPMN gateway routes on the variable (FRAMEWORK §2)	Declarative business rules per operator, reusable, owned by business analysts
Order journey expressible in English domain vocabulary	English state names: CAPTURED, IN_VALIDATION, AWAITING_PAYMENT, IN_KYC, READY_FOR_INSTALL, ACTIVATED, etc.	KE deployment needs English-default vocabulary; per-operator i18n where customer-facing
Decoupled order ↔ subscription lifecycle	customer_order is the orchestration aggregate (distinct from subscription); subscription_id is a column on customer_order linking the two. The orchestration runs to completion (ACTIVATED, CANCELLED, REJECTED) whether or not the subscription advances in parallel. STEP-PAYMENT correlates inbound payments via a side-channel mapping table keyed on order_id (see STEP-PAYMENT §1)	The order journey is its own first-class entity, with its own auditable lifecycle, separate from the resulting subscription

5.4 Migration approach

1. Deploy fulfillment-order-capture module with the schemas in FRAMEWORK, the workers across step satellites, the Kafka consumers, the initial operator BPMN (OrderCapture_WIK.bpmn).
2. Seed ful_02_config per operator (worker lock durations, retry policy, activity-phase map).
3. Configure Drools KJAR per operator for gateway decisions.

4. Configure per-operator user-task candidate groups in FOUNDATION_AUTH (kenya-l1-kyc, kenya-kyc-leaders). Future operators that introduce new user task types add their own candidate groups at deployment time.
5. Deploy v1.0 stubs (customer_stub, install_wo_stub, staff_notification_stub); retire as ILM-CFG-01, WO module, ICN-01 land. **WO module has shipped** (DD_WO-01-Work_Order_Module-v1.0); STEP-INSTALL retires install_wo_stub by switching to real POST /api/work-orders calls to the WO Installation flow.
6. Channel team integrations: Field Agent PWA, Backoffice UI, Call Center UI build against POST /api/orders and the user-task completion endpoints. The existing TZ frontend (whatever its form — to be confirmed in cartography) may be repurposable as the basis for the KE Field Agent PWA.
7. Self-service portal builds against the same API with ORDER_CUSTOMER_SELF auth.
8. **State translation** from any TZ historical-order import that the KE go-live plan decides to do: the mapping from TZ-codebase states → V3 phases will be defined during the cartography phase, once the actual TZ state model is known. V3 supports either path: backfill historical orders with synthetic process_instance_id (no live Camunda instance — they're history-only) and the corresponding customer_order_phase_history audit trail, or start fresh at cutover. Decision open.

Mapping to TZ baseline (§5.2) — what V3 adds vs what the redeployed codebase already supports, including the actual state translation table — will be filled in after codebase review.

5.5 Open coordination items handed off

- **ILM-CFG-01 Customer Service DD** (pending) — blocker for retiring customer_stub.
- **WO module DD** ■ shipped (DD_WO-01-Work_Order_Module-v1.0 + framework + 3 flow satellites). install_wo_stub is retired in STEP-INSTALL — STEP-INSTALL now calls POST /api/work-orders against the WO Installation flow directly (see STEP-INSTALL §3 for the contract).
- **ICN-01 Internal Communications DD** (pending) — blocker for retiring staff_notification_stub. Staff-facing counterpart to DD_NOT-01. Channels: email + Slack + Microsoft Teams + in-app push to BO UI.
- **Order-level SLA tracking** — V3 candidate is bringing SLA tracking inside SOPHIX (future FUL-OPS-SLA DD).
- **Real-time self-service portal** — replaces the legacy daily-refresh customer app.
- **Multi-DB topology** — open workshop question; V3 scope is WIK/WUG/WTZ/YASSN only.
- **Move WO linkage + manual resume rules** — workshop opens, don't block FUL-02 v1.0 (MACD out of scope).
- **Multi-tenant Camunda design decision** — FOUNDATION_CAMUNDA doesn't currently address per-operator tenancy. v1.0 uses per-operator process keys — compatible with future Camunda multi-tenancy if added.